

Git Revert Command

What is `git revert` ?

The `git revert` command is used to undo the changes introduced by a previous commit by creating a **new commit** that reverses those changes. It is the safest way to undo changes without modifying Git history.

 **Watch the Video:**

<https://youtu.be/Mv8AKqCCkp0?si=k3FjdTR0iSnwOBpx>

What You Will Learn

- What the `git revert` command is and why it is important.
- How to undo changes after a commit without affecting Git history.
- Practical scenarios where `git revert` is the preferred way to reverse changes.
- The difference between `git revert` and `git reset`.
- How `git revert` helps maintain a clean and reliable commit history.

Note: `git revert` creates a new commit to reverse previous changes, making it the recommended approach for shared and public repositories.

Git Revert Command

Introduction

The `git revert` command is used to **undo the changes made by a previous commit** by creating a **new commit** that reverses those changes. Unlike `git reset`, it **does not remove or rewrite commit history**, making it the safest option for undoing changes in shared repositories.

Syntax

```
git revert <commit-hash>
```

Example:

```
git revert a1b2c3d
```

Git creates a new commit that reverses the changes introduced by the specified commit.

Revert the Latest Commit

```
git revert HEAD
```

This command creates a new commit that undoes the changes from the most recent commit.

Revert Multiple Commits

```
git revert <oldest-commit>^..<latest-commit>
```

Reverts a range of commits by creating separate revert commits.

How `git revert` Works

Suppose your commit history looks like this:

```
A → B → C → D (HEAD)
```

If you run:

```
git revert C
```

Git creates a new commit:

A → B → C → D → E (HEAD)

Where **E** contains the opposite changes of **C**. The original commit **C** remains in the history.

When to Use `git revert`

- Undo a commit that has already been pushed to GitHub.
- Safely reverse changes without deleting commit history.
- Correct mistakes in shared or collaborative projects.
- Maintain a complete and reliable project history.

`git revert` VS `git reset`

<code>git revert</code>	<code>git reset</code>
Creates a new commit	Removes or moves commits
Preserves commit history	Rewrites commit history
Safe for shared repositories	Best for local repositories
Recommended after pushing commits	Recommended before pushing commits

Key Points

- Creates a **new commit** to undo previous changes.
- Does **not** delete existing commits.
- Preserves the complete Git history.
- Safe for team collaboration and public repositories.

- Preferred over `git reset` when commits have already been shared.

Best Practice: Use `git revert` when you need to undo changes in a repository that others are using. It keeps the project history intact while safely reversing unwanted changes.